Table 4.1 What an HTTP `OPTIONS` request and response might look like in a pre-CORS world

HTTP request	HTTP response

`OPTIONS /api/posts HTTP/1.1`
`User-Agent: Chrome`
`Host: 127.0.0.1:9999`
`Accept: */*`
	`HTTP/1.1 200 OK`
	`Allow: GET, POST`

> **NOTE** You may have noticed that the preflight request itself is a cross-origin request. This was deemed acceptable by the CORS spec authors, because the preflight request is used in the way `OPTIONS` requests are intended.

An `OPTIONS` request with an Origin header is not necessarily a preflight request. To distinguish a regular `OPTIONS` request from a preflight `OPTIONS` request, a preflight request will always contain an Access-Control-Request-Method header, as discussed next.

4.3.3 Access-Control-Request-Method header

The Access-Control-Request-Method request header asks the server for permission to make a request using a particular HTTP method. The preceding example of deleting a post uses the HTTP `DELETE` method. Therefore, the Access-Control-Request-Method header would be set to `DELETE`:

```
Access-Control-Request-Method: DELETE
```

The Access-Control-Request-Method header is always set to the value of the HTTP method for the actual request, as shown in table 4.2. Because an HTTP request must have an HTTP method, a preflight request must have an Access-Control-Request-Method header.

Table 4.2 Mapping the actual request method to the preflight

Preflight request	Actual request

`OPTIONS /data HTTP/1.1`
`User-Agent: Chrome`
`Host: localhost:10009`
`Accept: */*`
`Origin: http://localhost:10007`
`Access-Control-Request-Method: DELETE`
	`DELETE` `/data HTTP/1.1`
	`User-Agent: Chrome`
	`Host: localhost:10009`
	`Accept: */*`
	`Origin: http://localhost:10007`

To recap, a preflight request must have an HTTP `OPTIONS` method, and it must contain an Origin and Access-Control-Request-Method header. Now that you know what comprises a preflight request, let's modify the sample code to detect these three characteristics.